

Phase -1.11 — Capacity Planning & Scalability

Assumptions

Objective

Before implementing any infrastructure, it is important to estimate how the system is expected to behave under load.

Capacity planning helps answer questions such as:

- How many users should the system support?
- How many requests per second are expected?
- How much data will be stored?
- Which components become bottlenecks first?
- Which components should scale horizontally?
- Where should caching be introduced?
- Which database queries require indexing?

The goal is **not** to predict exact production traffic but to design a system that can scale gracefully as demand increases.

Why Capacity Planning?

Many architectural decisions depend on expected load.

Examples:

- Should Redis be introduced?
- Is connection pooling necessary?
- Do we need background workers?
- Should files be stored in the database?
- How many application instances should run?

Without understanding traffic assumptions, it is impossible to justify these decisions.

Assumptions

Since this is a learning project, we will define realistic assumptions inspired by a medium-sized ticket booking platform.

Users

Metric	Assumption
Registered Users	100,000
Daily Active Users	15,000
Concurrent Users	2,000
Peak Concurrent Users	10,000

Events

Metric	Assumption
Total Events	5,000
Active Events	500
Average Seats per Event	1,000
Largest Event	50,000 Seats

Bookings

Metric	Assumption
Bookings per Day	25,000
Peak Booking Requests	500 / second
Average Booking Duration	< 1 second
Seat Hold Timeout	10 minutes

Authentication

Metric	Assumption
Login Requests	5,000 / day
Refresh Token Requests	20,000 / day
Average User Sessions	3

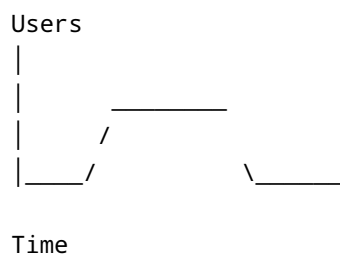
Traffic Patterns

Traffic is not evenly distributed.

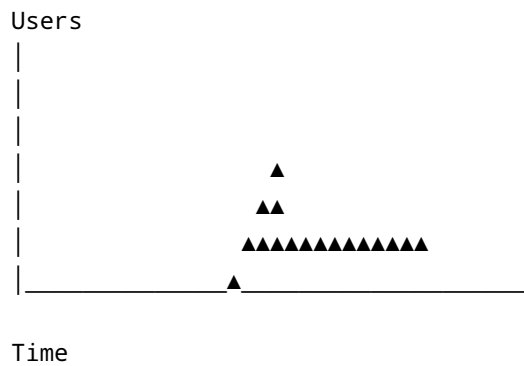
Most of the time, the system experiences moderate activity.

During ticket launches, demand increases dramatically.

Normal Day



Popular Event Launch



The architecture must perform reliably during these spikes.

Read vs Write Ratio

Most requests are reads.

Approximate ratio:

Read Requests : 90%

Write Requests : 10%

Examples of read operations:

- Browse events
- View seat layout
- View bookings

Examples of write operations:

- Register
- Login
- Create booking
- Payment

This justifies introducing Redis caching.

Storage Estimation

Estimated record counts after one year.

Table	Estimated Records
Users	100,000
Events	5,000
Seats	5,000,000
Bookings	2,000,000
Payments	2,000,000
Refresh Tokens	300,000

Seats occupy the largest table because every event owns many seats.

Database Growth

Expected growth:



Booking and payment tables grow continuously.

Historical records should never be deleted.

API Traffic Estimation

Approximate peak traffic.

Endpoint	Peak Requests/Second
GET /events	1,500
GET /events/{id}	800
GET /events/{id}/seats	1,200
POST /bookings	500
POST /payments	250
POST /auth/login	80
POST /auth/refresh	150

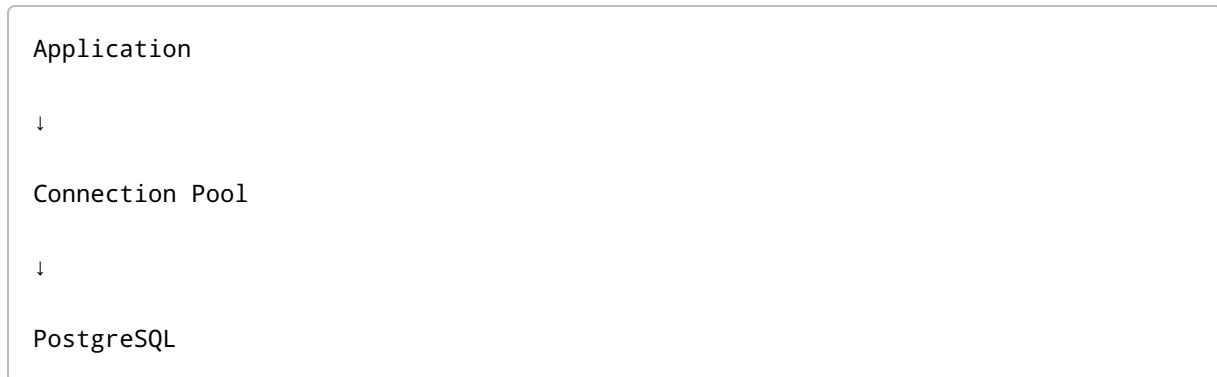
These estimates help determine where optimization is most valuable.

Database Connections

A database connection is expensive.

Each HTTP request should **not** create a new connection.

Instead:



Connection pooling improves performance and prevents exhausting database resources.

Prisma internally manages a connection pool, but understanding why pooling exists is essential.

Caching Strategy

Redis should cache only frequently accessed data.

Recommended cache candidates:

- Event list
- Seat layouts
- Venue information (future)

Do **not** cache:

- Authentication state
- Booking status
- Payment status

These values change frequently and require strong consistency.

Background Job Volume

Expected asynchronous jobs.

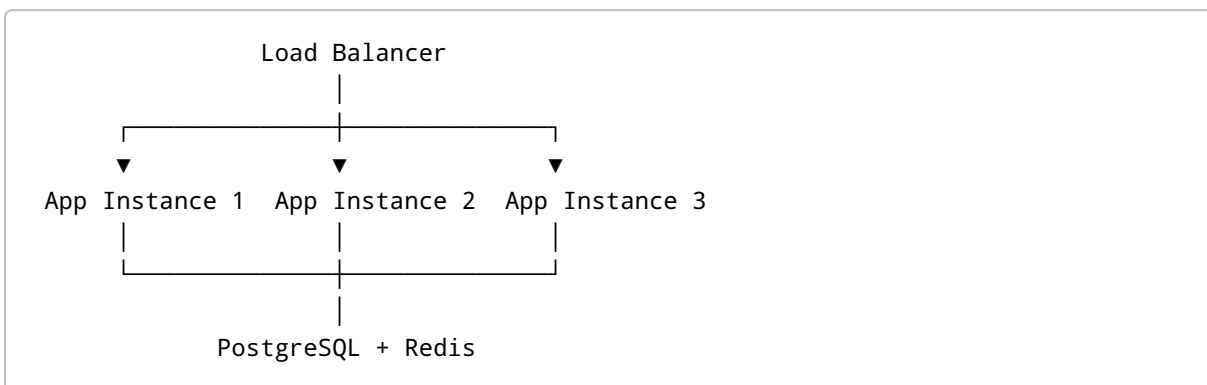
Job	Estimated Daily Volume
Seat Expiration	25,000

Job	Estimated Daily Volume
Booking Confirmation	25,000
Cleanup Tasks	5,000
Refresh Token Cleanup	20,000

These jobs should never block user requests.

Horizontal Scaling

The API layer should scale horizontally.



Every application instance must be stateless.

JWT authentication enables this design because session data is not stored in server memory.

Database Scaling

Version 1 uses a single PostgreSQL instance.

Future scaling options include:

- Read replicas
- Connection pooling proxies
- Table partitioning
- Query optimization
- Index tuning

Database sharding is intentionally excluded from this project.

Redis Scaling

Initially:

One Redis Instance

Future options:

- Redis Cluster
- Redis Sentinel
- Managed Redis Service

For this project, a single Redis instance is sufficient.

Bottleneck Analysis

Expected bottlenecks in order of likelihood.

1. PostgreSQL write throughput.
2. Seat booking concurrency.
3. Slow database queries.
4. Missing indexes.
5. Large seat layout queries.
6. Redis memory usage.
7. Background worker backlog.
8. Network latency.

Recognizing likely bottlenecks helps prioritize optimization efforts.

Performance Optimization Strategy

The system should optimize in the following order:

1. Correct algorithms.
2. Efficient database queries.
3. Proper indexing.
4. Connection pooling.
5. Redis caching.
6. Background processing.
7. Horizontal scaling.
8. Infrastructure upgrades.

Hardware should be the final optimization, not the first.

Failure Scenarios

The architecture should anticipate failures.

PostgreSQL Unavailable

Expected behavior:

- Return Service Unavailable.
 - Log the error.
 - Do not crash the application.
-

Redis Unavailable

Expected behavior:

- Continue serving requests.
- Fall back to the database.
- Disable caching temporarily.
- Queue-related functionality may become unavailable.

Redis should improve performance, not become a single point of failure.

Background Worker Offline

Expected behavior:

- Jobs remain in BullMQ.
- Worker resumes processing when restarted.

No jobs should be lost.

Application Instance Crash

Expected behavior:

- Other application instances continue serving traffic.
- Stateless architecture minimizes disruption.

Future Scaling Opportunities

The current architecture intentionally supports future enhancements.

Examples include:

- CDN for static assets.
- Read replicas.
- Distributed Redis Cluster.
- Message broker such as Kafka.
- Microservice decomposition.
- Kubernetes deployment.
- Auto-scaling groups.
- Multi-region deployments.

None of these are required for Version 1, but the architecture should not prevent adopting them later.

Capacity Planning Summary

Component	Initial Strategy	Future Strategy
API	Single Instance	Horizontal Scaling
PostgreSQL	Single Instance	Read Replicas
Redis	Single Instance	Redis Cluster
Background Jobs	Single Worker	Multiple Workers
Authentication	JWT	Same
Storage	Local Docker Volume	Managed Database

Deliverables

At the completion of this phase, the system has clearly defined operational assumptions.

The team understands:

- Expected traffic.
- Storage growth.
- Read/write patterns.
- Scaling strategy.

- Bottlenecks.
- Failure scenarios.
- Performance expectations.
- Infrastructure evolution.

These assumptions justify many of the architectural decisions made throughout previous phases and provide a reference point for future optimization.

The next phase, **Phase -1.12 — Git Strategy & Development Workflow**, will establish the repository structure, branching model, commit conventions, release strategy, code review workflow, GitHub Actions integration, and development lifecycle followed throughout the project.